Modeling and Control of a Dynamical System

Planar Drone (2D Quadrotor)

LQR Controller Design & Simulation

Control Theory — Project Report

Academic Year 2025/2026

Table of Contents

1. System Description

The planar drone (2D quadrotor) is a simplified model of a multirotor aerial vehicle constrained to move in a single vertical plane. It represents one of the canonical nonlinear control problems in robotics and aerospace engineering — combining translational and rotational dynamics with underactuation.

The physical system consists of a rigid body equipped with two rotors, each producing a vertical thrust force. The drone can translate horizontally (x) and vertically (y), and rotate about its center of mass (pitch angle θ). Because the system has two independent inputs (F_1 and F_2) but three degrees of freedom, it is underactuated — a key challenge that makes this system particularly interesting from a control perspective.

1.1 Physical Parameters

Parameter	Value	Description
m	1.0 kg	Quadrotor mass
g	9.81 m/s²	Standard gravitational acceleration
I	0.05 kg·m²	Estimated rotational inertia about z-axis
L	0.20 m	Half-span from CoM to each rotor (arm length)
F_max	15 N	Maximum thrust per rotor
F_min	0 N	Rotors cannot produce negative thrust
F_0 (hover thrust)	4.905 N	mg/2 per rotor at equilibrium

1.2 Assumptions

- The drone moves in a 2D (x,y) vertical plane — only up and down, no out-of-plane motion.
- The rigid body assumption, no structural deformation.
- Rotor thrust is directly proportional to a scalar control input; actuator dynamics are neglected.
- No aerodynamic drag is modelled (important limitation discussed in Section 4).
- The center of mass is in the same place as the geometric center of the drone body.
- Rotors are in the same distance from the center: arm length L = 0.20 m each side.

2. Mathematical Model

2.1 State Variables, Inputs and Outputs

The state vector contains six scalar quantities describing the full configuration and velocity of the drone:

$$\mathbf{q} = [x,\ \dot{x},\ y,\ \dot{y},\ \theta,\ \dot{\theta}]^T \in \mathbb{R}^6$$

i	State variable	Symbol	Unit	Description
1	Position x	x	m	Horizontal position in horizontal frame
2	Horizontal velocity	$\dot{x}$	m/s	Time derivative of x
3	Position y	y	m	Vertical position in vertical frame
4	Vertical velocity	$\dot{y}$	m/s	Time derivative of y
5	Pitch angle	θ	rad	Rotation about z-axis (positive CCW)
6	Angular rate	$\ddot{\theta}$	rad/s	Time derivative of θ

Control inputs: $\mathbf{u} = [F_1, F_2]^T$, where F_1 is the thrust of the right rotor and F_2 is the thrust of the left rotor. Both are bounded: $F \in [0, 15]$ N.

System outputs: $\mathbf{y} = [x, y, \theta]^T$ — the position and attitude, which we assume to be fully measurable.

2.2 Nonlinear Equations of Motion

Applying Newton's second law for translation and Euler's equation for rotation, the nonlinear equations of motion are derived:

Horizontal dynamics (x-axis)

$$m\ddot{x} = -(F_1 + F_2)\sin(\theta)$$

The horizontal acceleration is created by tilting the total thrust vector (sum of F1+ F2). At zero pitch (θ = 0), no horizontal force is generated — which is why the system is nonlinear and underactuated.

Vertical dynamics (y-axis)

$$m\ddot{y} = (F_1 + F_2)\cos(\theta) - mg$$

Vertical acceleration depends on the projection of total thrust onto the vertical axis, reduced by gravitational acceleration g. At hover, $(F_1 + F_2)\cos(\theta) = mg$.

Rotational dynamics (pitch)

$$I\ddot{\theta} = L(F_1 - F_2)$$

The net torque about the center of mass is proportional to the thrust difference between the two rotors, multiplied by the moment arm L. This equation is linear in the inputs.

Compact nonlinear state-space form

Defining the state vector q and control input $u = [F_1, F_2]^T$, the system can be written as:

$$\dot{q} = f(q, u)$$

where f is nonlinear due to the sin(θ) and cos(θ) terms connecting rotation with translation. The system has two derivatives with respect to each output, and it is set up in a way to allow us design a smooth paths (that enables trajectory generation).

2.3 System Classification

- Nonlinear: sin(θ) and cos(θ) connection between rotational and translational subsystems.
- Underactuated: 2 independent inputs (motors), 3 degrees of freedom — horizontal position cannot be controlled alone without impact of pitch (drone must tilt to move sideways).
- Linear in control: F_1 and F_2 appear linearly in the equations (not inside any nonlinear functions like sin, cos) what makes it possible to be controlled by the LQR.
- Easy path planning: we can plan the x(t) and y(t) trajectory first and then calculate the required tilt and thrust from it.

2.4 Equilibrium (Hover) Point

The equilibrium is in our case the hover state (stabilization), what is the configuration in which the drone is stationary with zero pitch:

$$\mathbf{q^* = [x_0,\ 0,\ y_0,\ 0,\ 0]^T}$$

$$\mathbf{u^* = [F_0,\ F_0]^T \quad where \quad F_0 = mg/2 = 4.905\ N}$$

This equilibrium exists for any horizontal position x_0 and any height y_0 — the drone is stable in horizontal and vertical position at hover. And in order to be stable in vertical position the sum of both forces F_1 and F_2 generated by the motors must be equal and compensate the gravitational force. Having those defined the LQR controller will stabilise deviations from a chosen reference point.

3. Linearisation Around Hover

Having the equilibrium defined, we can now design a controller that keeps the drone near this hover condition. We chose LQR (Linear Quadratic Regulator) - an optimal control method that balances peformance and motor effort. However LQR applies to linear systems, therefore we firstly have to linearise the nonlinear dynamics around the hover, which is the equilibrium point. Defining deviations as variables:

$$\mathbf{\delta q = q - q^*, \quad \delta u = u - u^*}$$

Where q* is the vector of approximated q's being the hover state $[x_0,\ 0,\ y_0,\ 0,\ 0]^T$, and u* is the vector of approximated inputs u's being the hover thrusts $[F_0,\ F_0]^T = [4.905\ N,\ 4.905\ N]^T$.

and applying first-order Taylor expansion (sin(θ) ≈ θ, cos(θ) ≈ 1 for small θ), the linearised system is:

$$\mathbf{\delta \dot{q} = A\,\delta q + B\,\delta u}$$

3.1 State Matrix A

The Jacobian ∂f/∂q evaluated at the equilibrium gives:

$$\mathbf{A = \partial f/\partial q|_{\{q^*,u^*\}}}$$

Where $f = \dot{q}$

$$q = [\overset{q_1}{x}, \overset{q_2}{\dot{x}}, \overset{q_3}{y}, \overset{q_4}{\dot{y}}, \overset{q_5}{\theta}, \overset{q_6}{\dot{\theta}}]^T$$

$$\dot{q} = [\dot{x}, \ddot{x}, \dot{y}, \ddot{y}, \dot{\theta}, \ddot{\theta}]^T$$

$q_1 = x$
$\dot{q_1} = \dot{x} = q_2$
$q_3 = y$
$q_4 = \dot{y} = \dot{q_3}$
$q_5 = \theta$
$q_6 = \dot{\theta} = \dot{q_5}$

$$\dot{q_1} = q_2$$
$$\dot{q_2} = \ddot{x} = \frac{-(F_1+F_2)\cdot\sin(\theta)}{m} = -\frac{(u_1+u_2)\cdot\sin(q_5)}{m}$$

$$\dot{q_3} = q_4$$
$$\dot{q_4} = \ddot{y} = \frac{(F_1+F_2)\cdot\cos(\theta)}{m} - g = \frac{(u_1+u_2)\cdot\cos(q_5)}{m} - g$$

$$\dot{q_5} = q_6$$
$$\dot{q_6} = \ddot{\theta} = \frac{L(F_1-F_2)}{I} = \frac{L(u_1+u_2)}{I}$$

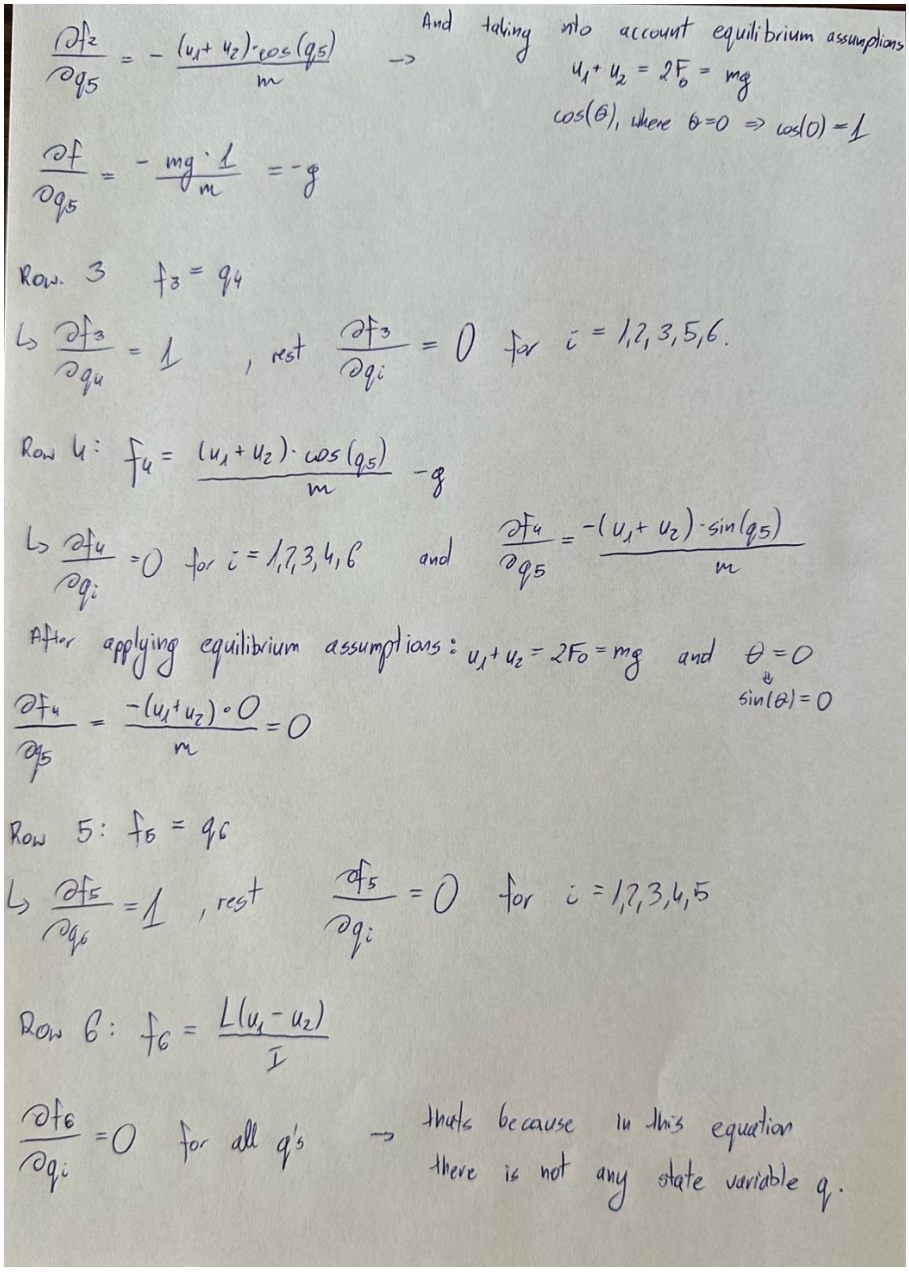
$$A = \frac{\partial f}{\partial q}$$

Row 1: $f_1 = q_2$

↳ $\frac{\partial f_1}{\partial q_1} = 0$ $\frac{\partial f_1}{\partial q_2} = 1$ $\frac{\partial f_1}{\partial q_3} = 0$ $\frac{\partial f_1}{\partial q_4} = 0$ $\frac{\partial f_1}{\partial q_5} = 0$ $\frac{\partial f_1}{\partial q_6} = 0$

Row 2: $f_2 = -\frac{(u_1+u_2)\cdot\sin(q_5)}{m}$

↳ $\frac{\partial f_2}{\partial q_1} = 0$ and ∂f_2 over q_2, q_3, q_4, q_6 will be equal to 0.



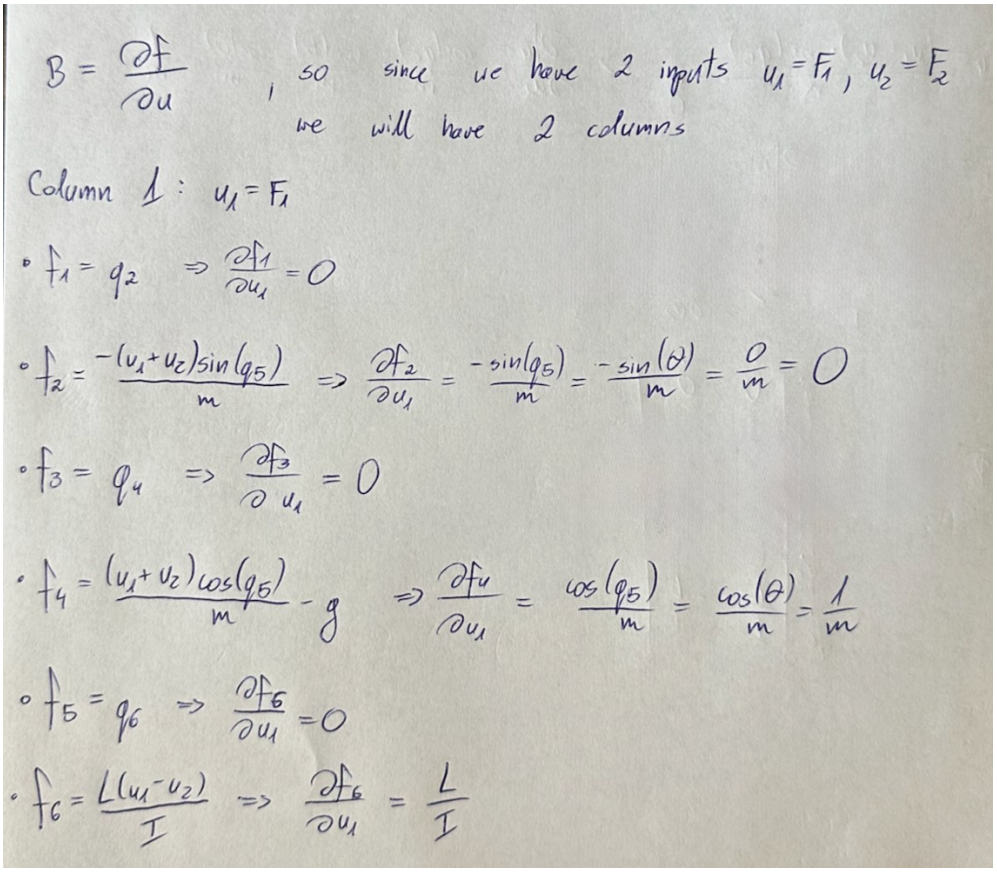
$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -g & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

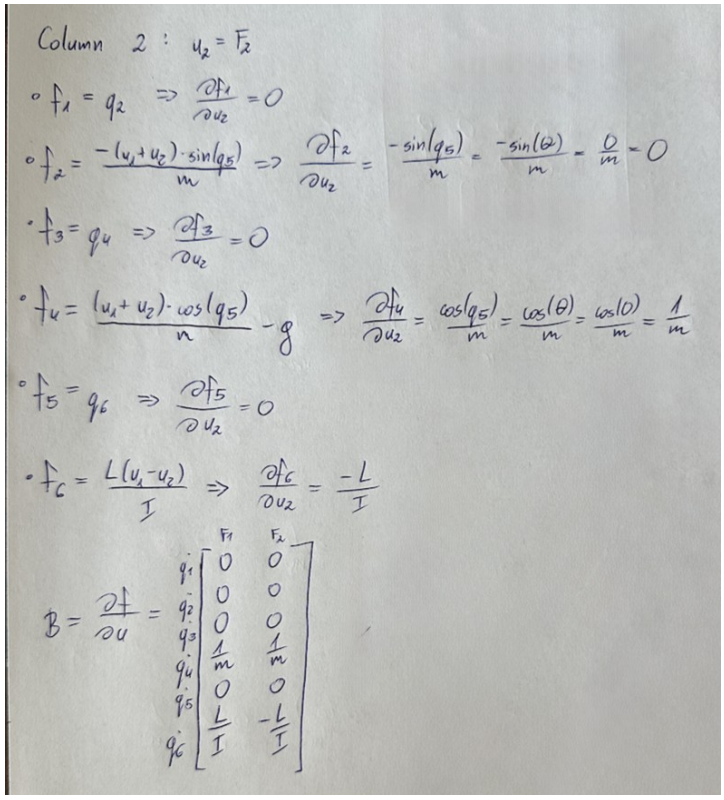
The −g entry in row 2, column 5 captures the horizontal acceleration induced by a pitch perturbation: a small tilt θ causes a horizontal force $-(2F_0)\sin(\theta) \approx -mg\cdot\theta$.

3.2 Input Matrix B

The Jacobian ∂f/∂u evaluated at the equilibrium gives:

$$\mathbf{B = \partial f/\partial u}|_{\{q^*,u^*\}}$$





$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/m & 1/m \\ 0 & 0 \\ L/I & -L/I \end{bmatrix}$$

Row 4 (vertical acceleration) responds equally to both rotors. Row 6 (angular acceleration) responds antisymmetrically — differential thrust creates torque.

3.3 Controllability

The controllability matrix S = [B, AB, A^2B, A^3B, A^4B, A^5B] was verified to have full rank (rank 6). The linearised system is therefore fully controllable — any state can be reached from any initial condition in finite time, confirming that LQR can be applied.

4. LQR Controller Design

4.1 Method Selection and Justification

Linear Quadratic Regulator (LQR) was selected as the control method for the following reasons:

- The linearised system is fully controllable, satisfying the necessary condition for LQR applicability.
- LQR gives guaranted closed-loop stability for the linearised system and good performance near the operating point.
- It returns an optimal state feedback gain K by minimising a quadratic cost — so giving a clear trade-off between performance and control effort.
- The method is computationally great, the algebraic Riccati equation is solved just once before the simulation starts, the controller only does a simple multiplication K*error- so it's very fast and easy to implement in real time.

The idea of LQR is based on the control law looking as follows:

$$\delta u = -K\delta q$$

Which is a linear feedback of state- each of the state variables so $\mathbf{[x,\ \dot{x},\ y,\ \dot{y},\ \theta,\ \dot{\theta}]^T}$ is scalled by an appropriate k factor from the matrix K and summed by the controller.

4.2 Optimal Control Problem

The LQR minimises the following cost function over infinite time:

$$\mathbf{J} = \int_0^\infty \left[\delta q^T \ Q \ \delta q \ + \ \delta u^T \ R \ \delta u \ \right] dt$$

where $\mathbf{Q} \in \mathbb{R}^{6x6}$ (positive semi-definite) penalises state error, and $\mathbf{R} \in \mathbb{R}^{2x2}$ (positive definite) penalises control effort.

- Part: $Q\delta q$ – penalasises state error (position, velocity, angle errors, when curernt state is not equal to desired state).
- Part: $R \ \delta u$ - penalises for using too much force (energy usage, motors overload)

The Q and R digonal matrices are chosen with appropriate weights. The higher the weight on a selected variable in Q matrix, the higher importance and effort of the controller to try to keep the error of this variable close to zero. The higher the weight in R matrix the more efficient use of motors is provided.

4.3 Cost Matrix Tuning

The diagonal matrices Q and R were tuned as follows:

$$\mathbf{Q = diag(20, 2, 20, 2, 15, 1)}$$

$$\mathbf{R = diag(0.5, 0.5)}$$

Interpretation of the Q weights:

- Position error (x, y): weight 20 — high priority on position accuracy (being in the right place).
- Velocity error ($\dot{x}$, $\dot{y}$): weight 2 — moderate damping of translational motion (provides smooth movement, no sudden jumps).
- Pitch angle (θ): weight 15 — strong priority on maintaining near-zero tilt (for safety reasons and to allow the approximations necessary for linearity computations).
- Angular rate ($\dot{\theta}$): weight 1 — light damping of rotation.

The R weights of 0.5 allow moderate controling force, preventing the rotors from excessively aggressive actions while still achieving the target fast enough.

4.4 Riccati Equation and Gain Matrix

The optimal gain matrix K is obtained by solving the continuous-time algebraic Riccati equation (CARE):

$$\mathbf{A^T P + P A - P B R^{-1} B^T P + Q = 0}$$

By sloving this equation the P matrix is calculated numerically, we obtained this by using the following command in python:

(`scipy.linalg.solve_continuous_are`)

Then the optimal feedback gain is:

$$\mathbf{K = R^{-1} B^T P} \quad \in \mathbb{R}^{2x6}$$

Solved numerically via

```
K = np.linalg.inv(R) @ B.T @ P
```

the resulting gain matrix is:

$$K = \begin{bmatrix} -4.472 & -3.608 & 4.472 & 2.544 & 12.081 & 2.005 \\ 4.472 & 3.608 & 4.472 & 2.544 & -12.081 & -2.005 \end{bmatrix}$$

4.5 Control Law

The full control law in terms of physical rotor thrusts is:

```
error = state - goal_state
delta_u = K @ error
F1 = F0 - delta_u[0]
F2 = F0 - delta_u[1]
F1 = np.clip(F1, F_min, F_max)
F2 = np.clip(F2, F_min, F_max)
```

Firstly the error of each variable is calculated by substracting the vector of goal state of all variables from the vector of current states of all variables:

$$Error = Current\ state - Goal\ state$$

Later on this error vector is multiplied by the gain matrix K which corresponds to the aproppriate variable in the error vector and it will give us the δu (1x2) vector, which consists of 2 values being the summed errors for each rotor: $\delta u[0]$ for the F_1 anfd $\delta u[1]$ for the second one F_2.

$$\delta u = K * error$$

The important part is that the K values (the gains) were calucalted accordingly to Riccati equation and those values of K correspond to each variable is appropriately high. It will ensure that when an important variable in the system, like for e.g. θ (which is 5-th collumn), will be far from the reference value it will be multiplied by the biggest gain in the matrix, what will result in a huge error obtained in the δu.

$$\mathbf{u = u^* - K (q - q_{ref})}$$

$$\mathbf{u = u_{ref} - \delta u}\ \textit{(in code)}$$

$$\mathbf{F_1 = F_0 - K_1 (q - q_{ref}) = F_0 - \delta u[0]}$$

$$\mathbf{F_2 = F_0 - K_2 (q - q_{ref}) = F_0 - \delta u[1]}$$

where K_1 and K_2 are the first and second rows of K respectively,
$q_{ref} = [x_{ref}, 0, y_{ref}, 0, 0, 0]^T$
is the reference hover state, and $F_0 = mg/2$ is the nominal hover thrust. Rotor thrusts are clamped to the physical bounds [0, 15] N before being applied.

And this part in the code is ensured by:

```
F1 = np.clip(F1, F_min, F_max)
F2 = np.clip(F2, F_min, F_max)
```

The clip function makes sure that if force of any rotor would exceed 15N, then the value will be cut on the 15N, and the lower bound which is 0N, also will not be exceeded.

Taking everything in the consideration we can see that at each time step the value of the rotor thrust is calculated to ensure the appropriate behavior of the drone in order to reach the target position.

4.6 Numerical integration (RK4)

To correctly simulate the drone's behavior it is necessary to correctly integrate the system. The nonlinear equations of motion are integrated numericaly using the 4- th order Ruge Kutta method with fixed timestep dt (in our case dt=0.005s).

At each simulation step, the LQR controller firstly computes the required rotor thrusts *u* based on the current state q(t). Then the RK4 takes over and approximates how the state evolves over the next dt seconds, assuming constant *u*. The RK4 evaluates the state derivative f(q, u) four times (at 4 points) per step:

$$k_1 = f(q,\ u) \rightarrow start\ \ of\ \ the\ \ interval$$

$$k_2 = f\left(q\ +\ \frac{dt}{2} * k1,\ \ u\right) \rightarrow midpoint\ \ estimate\ \ 1$$

$$k_3 = f\left(q + \frac{dt}{2} * k2,\ \ u\right) \rightarrow midpoint\ \ estimate\ \ 2$$

$$k_4 = f(q + dt * k_3,\ \ u) \rightarrow end\ \ of\ \ interval$$

The next state is then calculated as a weighted average as following:

$$q(t+dt) = q(t) + \frac{dt}{6} * (k_1\ +\ 2k_2 +\ 2k_3 +\ k_4)$$

The middle points k_2 and k_3 have the highest weight because they are important- they give the best trajectory approximation.
The RK4 was chosen over Euler approximation because it presents significantly higher accuracy at the same timestep size.

4.7 Closed-Loop Stability

The closed-loop system matrix A_cl = A − BK has eigenvalues:

$$\lambda_1 = -6.95, \quad \lambda_{2,3} = -2.66 \pm 2.52j$$

$$\lambda_4 = -3.76, \quad \lambda_{5,6} = -2.54 \pm 1.57j$$

All six eigenvalues lie in the open left half-plane (Re(λ) < 0), confirming asymptotic stability of the closed-loop linearised system and making sure that the system will work correctly. The complex conjugate pairs correspond to damped oscillatory modes in the coupled horizontal-pitch dynamics.

4.8 Limitations of the LQR Controller

Although the LQR controller performs well in simulation, several important limitations must be acknowledged:

- Small-angle validity: The controller was designed on a linearised model valid only for small pitch angles ($|\theta| < \sim 30°$). For large disturbances or aggressive manoeuvres the linearisation assumption breaks down and the controller may behave unexpectedly, as demonstrated in Scenario 3 where θ reached 48°.
- No angle wrapping: The controller operates on the raw value of θ without normalisation to $[-\pi, \pi]$. For initial angles where the shortest rotation would be in the opposite direction, the controller always rotates through the longer arc. A practical implementation should normalise the angle error before applying the control law.
- No integral action: LQR is a pure state-feedback controller with no integrator. In the presence of constant disturbances (e.g. wind) or model parameter errors, it will produce a steady-state offset. An LQI (LQR with integral term) or disturbance observer would be needed to eliminate this.
- Full state feedback assumption: LQR requires all six state variables to be known at every timestep. In practice, velocities ($\dot{x}$, $\dot{y}$, $\dot{\theta}$) are not directly measured and must be estimated — adding noise and delay to the feedback loop.
- Fixed gain: The gain matrix K is computed once offline for a single operating point (hover). If the drone's mass or inertia changes (e.g. payload), the controller is no longer optimal and may become unstable without retuning.

5. Simulation Results

Three simulation scenarios were run on the full nonlinear model using a Runge-Kutta 4th order (RK4) integrator with timestep dt = 0.005 s. The LQR controller was computed on the linearised model and applied to the nonlinear dynamics.

5.1 Scenario 1 — Stabilisation

Initial state: x = 2 m, y = 1 m, θ = 0.3 rad. Reference: origin (0, 0). The drone is released from a displaced, tilted position and must return to hover at the origin.

- The position error is driven to zero within approximately 3 seconds.
- The pitch angle overshoots briefly (expected due to the coupling: to move left, the drone must first tilt right), then to stop it must react by tilting left to slow down and balance out the previous momentum after this the drone settles.
- Rotor thrusts differ significantly in the transient phase to generate the corrective torque allowing for movement at an angle, then converge to the hover value $F_0 = 4.905$ N.

  To firstly move left we can see that the F_1 force (of the right rotor) in the beginning is higher than the force F_2 (left rotor) and sum of those two forces is lower than 9,81N what means that the right rotor of a drone produces more force then the left one causing it to tilt left and allowing for movement to the left so in the direction of the origin, in the same time the sum of F_1 and F_2 being lower than 9,81N (gravitational force) causes the drone to lower its hight. Later the force F_2 (left rotor) increases to counter the momentum of movement to the left, caused by the previous force, it's done in order to tilt the drone to the right so that the theta lowers and the drone doesn't exceed the origin in horizontal frame. Later on the differenting F1 and F2 is just the stabilization of the drone, to converge the theta to the 0° angle (also the sum of F1 and F2 is bigger than gravitational force (at around 1s) since as we can see here the y state went bellow 0 so the drone has to fly up a little bit) and then both rotors produce the force desired in the hover value $F_0 = 4.905$ N, since the drone is stabilized.

5.2 Scenario 2 — Trajectory Tracking (Step Reference)

Initial state: hover at origin. Reference: (3 m, 2 m). The controller must drive the drone to a new position 3m to the right and 2m higher.

- The drone tilts to the right (negative θ) to accelerate horizontally, then corrects the momentum cause by the previous action and attitude as it approaches the target — classic coupled behaviour.
- Both x and y reach their reference values within ~2.5 seconds, with small overshoot (~7%). However the θ value stabilizes a little bit after 3s.
- Rotor thrusts peak during the acceleration phase, staying within the [0, 15] N bounds throughout esnures us that the saturation works correctly.

5.3 Scenario 3 — Disturbance Rejection

The drone hovers at the origin. At t = 2s, an impulsive disturbance is applied: $\dot{x}$ += 3 m/s, $\dot{\theta}$ += 4 rad/s.

- The LQR controller successfully rejects the disturbance, returning the drone to hover within ~3 seconds.
- Although peak angular deviation is approximately 48°, which exceeds the typical small angle approximation range , the controller still succesfully stabilises the drone. It works because controller immediately reduces the angle to small values, spending only a short time in the nonlinear region. This shows that the system is maintaining stability beyond the theoretical linearisation range.
- This simulation demonstrates robustness to impulsive external perturbations.

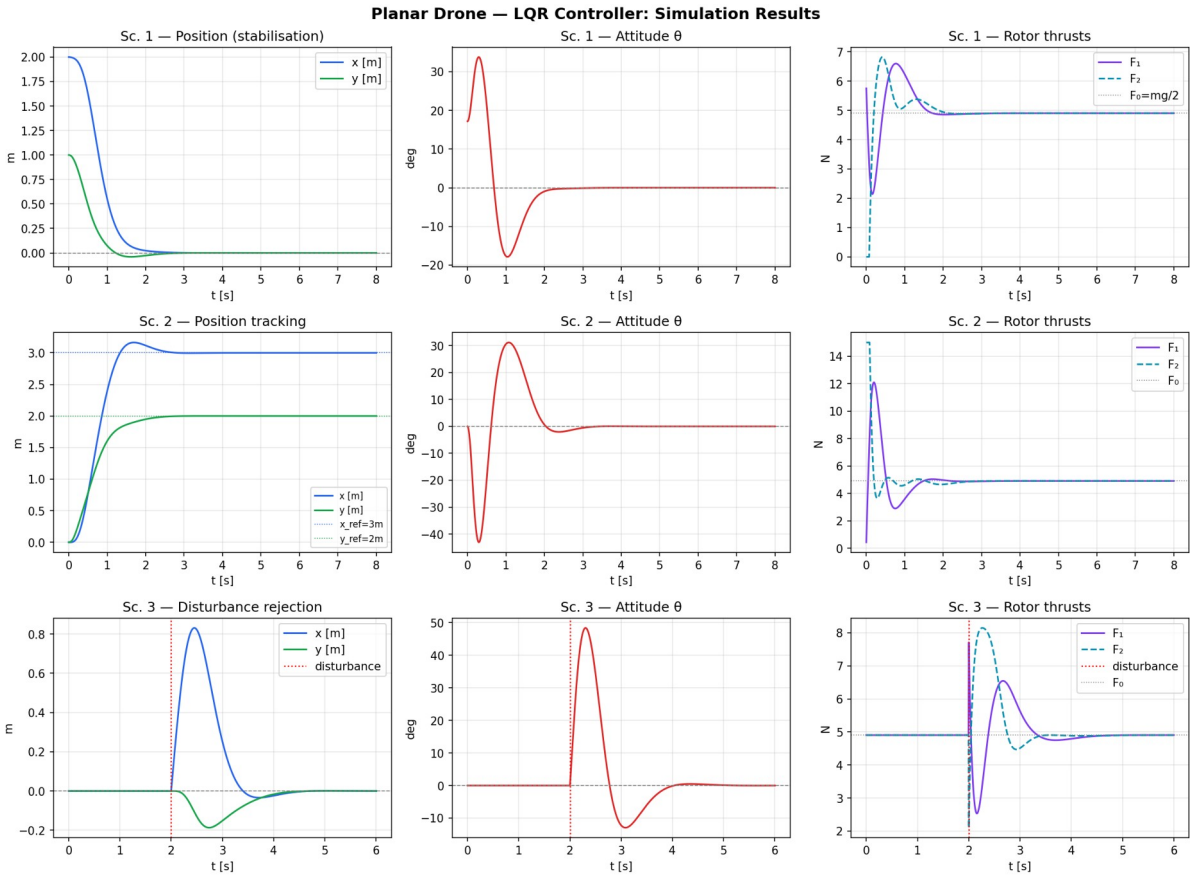


Figure 1. Time histories for all three scenarios: position (x, y), pitch angle θ, and rotor thrusts (F_1, F_2).

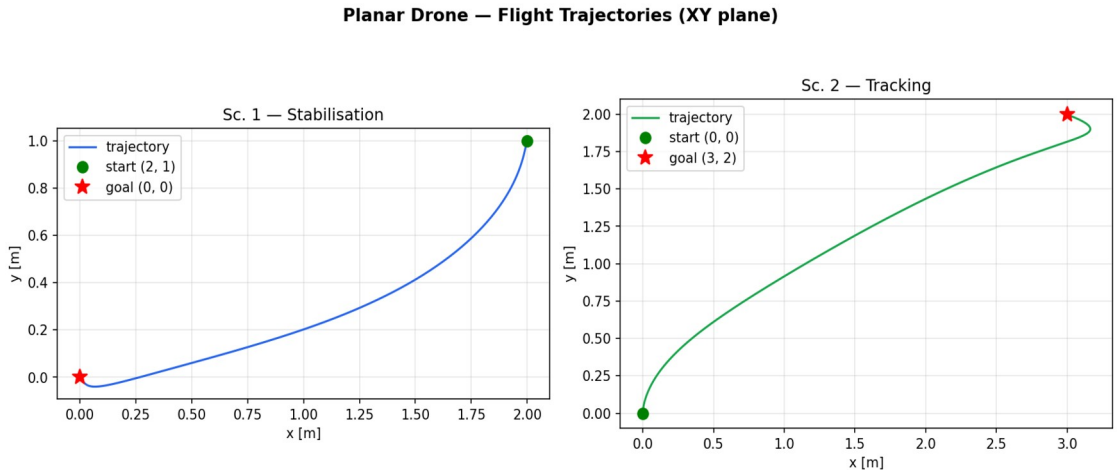


Figure 2. XY flight trajectories: stabilisation (left) and trajectory tracking (right).

5.4 Graphical Simulation — PyGame

In addition to the static matplotlib plots, a real-time interactive simulation was implemented using PyGame. The simulation runs the same nonlinear dynamics and LQR controller as the Python script, but renders the drone visually at 60 fps and allows the user to interact with the system in real time.

Implementation details:

- Physics: identical nonlinear dynamics (Newton-Euler equations) integrated with RK4 at dt = 1/240 s (4 substeps per frame), ensuring accurate simulation at interactive framerates.
- Controller: the same LQR gain matrix K computed offline from the Riccati equation. At each frame the controller evaluates $u = u^* - K \cdot \delta q$ and sends F_1, F_2 to the dynamics.
- Visualisation: the drone body and rotor arms are drawn as rotated rectangles. Thrust flames are rendered with length proportional to the current rotor force, making it easy to see when saturation (F = 15 N) is active. A trail shows the recent flight path.
- State panel: a side panel displays all six state variables ($x, \dot{x}, y, \dot{y}, \theta, \dot{\theta}$), current rotor thrusts F_1 and F_2, and simulation time — updated every frame.

Interactive controls:

- Mouse click — sets a new goal position. The drone immediately reacts and flies to the clicked point, demonstrating real-time trajectory tracking.
- Key R — resets the drone to the initial displaced state (x=2 m, y=1 m, θ=0.3 rad), allowing repeated observation of the stabilisation scenario.
- Key D — applies a random impulsive disturbance (random kick to $\dot{x}$ and $\dot{\theta}$), equivalent to Scenario 3, showing disturbance rejection live.
- SPACE — pauses and resumes the simulation.

The simulation can be run locally with: pip install pygame numpy scipy, followed by python drone_pygame.py. It provides an intuitive way to explore the controller behaviour beyond what static plots can convey — in particular the coupling between pitch angle and horizontal motion is immediately visible when a new goal is clicked.

6. Practical Feasibility

While the simulation demonstrates excellent performance, several practical issues must be addressed before implementing this controller in a physical system.

6.1 Actuator Limitations

- Real brushless DC motors have finite bandwidth (typically 10–50 Hz) — rapid and sudden thrust commands cause tracking lag not captured in the model.
- Electronic speed controllers (ESCs) introduce additional delays of 5–20 ms per command cycle.
- Minimum thrust is typically non-zero due to motor inertia; the F_min = 0 assumption used in our model is idealistic and it would not allign with real world reality.
- Battery voltage drop under high load reduces maximum available thrust over time.

6.2 Measurement Noise and State Estimation

- The LQR law requires full state feedback: $[x, \dot{x}, y, \dot{y}, \theta, \dot{\theta}]$. In practice, velocities are not directly measured and must be estimated from noisy sensor data.
- IMU accelerometers have small drift and noise (typically $\sigma \sim 0.01$ m/s²); using them to estimate velocity makes the errors accumulate over time.
- A Kalman Filter (KF) or Extended Kalman Filter (EKF) would be required in practice to fuse IMU and position sensor data with a model presented in this approach.

6.3 Sampling Frequency

- The controller must run at a sufficiently high sampling rate. Given closed-loop bandwidth of ~6 rad/s, a minimum sampling rate of ~60 Hz is required by Shannon's theorem; in practice 200–500 Hz is typical for drone control loops.
- Lower sampling rates could introduce discretisation effects that can degrade stability margins.

6.4 Model Mismatch

- Aerodynamic drag (in euqation uses squared velocity) was neglected — relevant at speeds above ~2 m/s.
- Ground effect (increased lift near the ground) is not modelled, additionaly the drone in our model can go bellow the ground.
- Parameter uncertainty (actual m, I, L may differ from nominal values) affects LQR performance; gain margin analysis would be needed in order to ensure the system works correctly.

6.5 Safety Considerations

- The underactuated nature (only two rotors (2 inputs) for three motions x,y, θ) means the drone cannot recover from all possible failure modes (e.g., single rotor failure will lead to the crush of the drone).
- Hardware safety limits that are not included in this simulation (like arming/disarming logic- so the motors don’t start unexpectedly, geofencing- keeping the drone inside a safe area, failsafe modes- what happens when the signal is lost) are essential for physical operation.
- The LQR validity is limited to small-angle perturbations ($|\theta| < \sim 30°$); large disturbances pushes the drone beyond this range, the linear controller may not be able to stabilise it. In such a cases, a nonnlinear controller would be needed.

7. Real-World Implementation

7.1 Required Sensors

- IMU (Inertial Measurement Unit): 3-axis accelerometer + gyroscope — measures $\dot{\theta}$ and body accelerations. Example: MPU-6050, ICM-42688-P.
- Barometer: to measure the vertical position y (absolute altitude). Example: BMP390. Accuracy ~0.1 m.
- Optical flow sensor or external motion capture: needed to measure horizontal position x, especially indoors, where GPS doesn't work. Example: PX4FLOW, OptiTrack.
- Magnetometer: measures heading (yaw). It is less important in our 2D model, but useful in 3D applications. Example: HMC5883L.

7.2 Required Actuators

- 2× Brushless DC motors with propellers: one motor per rotor, generating variable thrust to control the drone.
- 2× Electronic Speed Controllers (ESCs): take commands from flight controller- PWM signals (or DSHOT protocol) and convert them into motor speed- RPM.
- Propeller selection: the size- diameter and pitch determine how much thrust motor produces for a given speed. The propelers must be matched to motor's KV rating and the battery voltage.

7.3 Computing Hardware

- Microcontroller for inner loop- handles attitude (angle) control at around 500 Hz. Examples: STM32F4 or STM32H7 (the same chips used in Betaflight and PX4 autopilots).
- Companion computer for high level taks- outer loop (position, trajectory control) and state estimation. Examples: Raspberry Pi 4, NVIDIA Jetson Nano, or similar ARM-based SBC (single board computer).
- The LQR gain matrix K is computed offline and stored as a constant 2×6 matrix- it is computationally cheap; the online (during flight) computation is a single matrix-vector multiplication — extremely fast (< 1 microsecond on any modern microcontroller).

7.4 Additional Implementation Challenges

- Propeller wash and vibration affect IMU readings — real drones need mechanical vibration isolation and software notch filters to clean up the sensor data.
- The 2D model must be realised in 3D setup: to make a real drone move only in a plane, either a physical rail/slider mechanism is needed or a very robust full 3D controller treating the 2D dynamics as a special case.
- Real-time communication between sensors, microcontroller, and companion computer introduces jitter and latency. These delays must be kept within acceptable limits.
- Motors calibration and ESC arming procedures are needed before flight.

8. Conclusions

This project presented the complete modelling and LQR control design for a 2D planar drone. The nonlinear Newton-Euler equations of motion were derived, and the system was linearised around the hover equilibrium to obtain a controllable LTI state-space model. An LQR controller was designed by solving the algebraic Riccati equation, resulting in a gain matrix that places all closed-loop poles in the left half-plane.

Numerical simulations on the full nonlinear model confirmed successful stabilisation from a displaced initial condition, tracking of a step reference position, and rejection of an impulsive disturbance — all within physically realistic actuator bounds.

The main limitations of the approach are: validity only for small pitch angles (linearisation assumption), neglect of aerodynamic drag and actuator dynamics, and requirement for full state feedback (requiring a state estimator in practice). Future work could extend the controller to a full nonlinear method (feedback linearisation or MPC) to handle large-angle manoeuvres, or incorporate an LQG (LQR + Kalman Filter) structure to handle measurement noise.

References

[1] Murray, R. M., Li, Z., Sastry, S. S. (1994). A Mathematical Introduction to Robotic Manipulation. CRC Press.

[2] Anderson, B. D. O., Moore, J. B. (1990). Optimal Control: Linear Quadratic Methods. Prentice-Hall.

[3] Mahony, R., Kumar, V., Corke, P. (2012). Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor. IEEE Robotics & Automation Magazine, 19(3), 20–32.

[4] Tedrake, R. (2023). Underactuated Robotics: Algorithms for Walking, Running, Swimming, Flying, and Manipulation. MIT OpenCourseWare. http://underactuated.mit.edu

[5] Siciliano, B., Sciavicco, L., Villani, L., Oriolo, G. (2009). Robotics: Modelling, Planning and Control. Springer.

[6] scipy.linalg.solve_continuous_are — SciPy documentation. https://docs.scipy.org/doc/scipy/